

EMERGENCY SHEET — read only this

If you have a few hours, learn **just this page**. It covers the traps (easy MCQ/TF marks) and the 4 theory answers most likely to appear.

PART 1 — THE 13 TRAPS (near-guaranteed MCQ / True-False marks)

1. **SQA does NOT run the program. SQC does.** (SQA = prevent; SQC = detect)
 2. **CMM Level 4 = Managed, Level 5 = Optimizing.** (never reversed)
 3. **Security is NOT an ISO-9126 characteristic.** (only 6: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability)
 4. **Reliability = a FACTOR. Modularity = a CRITERION.**
 5. **Test Plan = high-level. Test Case = low-level.**
 6. **Regression testing REUSES old tests — designs NO new ones.**
 7. **Testing shows presence of defects, NEVER proves their absence.** (Principle 1)
 8. **Automation CANNOT replace manual testing.** (usability can't be automated)
 9. **Defect prevention is NEVER 100% effective.**
 10. **NVP and auto-save = CONTAINMENT, not prevention.**
 11. **Hazard = pre-condition to an accident. Accident = failure with severe consequences. Safety = accident-free.**
 12. **Static analysis = no execution. Dynamic analysis = executes the program.**
 13. **Error = human action. Fault = wrong code. Failure = system can't perform. Defect = deviation from spec (= "bug").**
-

PART 2 — LISTS TO MEMORIZE (with mnemonics)

- **CMM 5 levels** — "I Really Don't Manage Overtime": **Initial, Repeatable, Defined, Managed, Optimizing**
- **TMM 5 levels** (by Ilene Burnstein): **Initial, Phase Definition, Integration, Management & Measurement, Optimization**
- **7 testing principles** — "Please Eat Every Delicious Pizza Carefully, Alright": **Presence-not-absence, Exhaustive-impossible, Early testing, Defect clustering, Pesticide paradox, Context dependent, Absence-of-errors fallacy**
- **6 ISO-9126** — "Fat Rabbits Usually Eat More Peas": **Functionality, Reliability, Usability, Efficiency, Maintainability, Portability**
- **5 views of quality** (Kitchenham & Pfleeger) — "My Uncle Makes Pretty Vases": **Mystical, User, Manufacturing, Product, Value-based**
- **3 QA categories:** **Prevention** (before injection), **Reduction** (remove faults), **Containment** (limit failures)
- **4 testing levels + who:** **Unit → Developer, Integration → Tester, System → Tester, Acceptance → End-user**

2 incidents: **Therac-25** (radiation machine, killed patients, software bug) · **Ariane 5** (rocket exploded 37 sec after launch, 1996, most expensive software bug).

PART 3 — THE 4 THEORY ANSWERS (this is the 40-mark section)

If asked SQA vs SQC:

Definition: SQA = planned, process-oriented activity that **prevents** defects (Verification). SQC = product-oriented activity that **finds** defects by executing the product (Validation). 5 differences: prevent vs detect · no execution vs

executes · all team vs testing team only · verification vs validation · prevention system vs detection system. Example: writing the coding standard = SQA; running the login tests = SQC.

If asked Error / Fault / Failure / Defect:

Error = human action producing a wrong result → injects a **Fault** = wrong step/data in the code → when run, causes a **Failure** = system can't do its function. All three together = **Defects** ("bugs"). Example: a programmer mistypes a formula (error) → wrong line of code (fault) → app shows wrong total (failure).

If asked the 5 views of quality (Kitchenham & Pfleeger):

1. **Mystical** – quality recognized by experience, can't be defined.
2. **User** – meets user needs; good if it satisfies many users.
3. **Manufacturing** – conformance to requirements, "right the first time" (CMM & ISO 9001).
4. **Product** – good internal properties → good external properties.
5. **Value-based** – willingness to pay; excellence + worth; cost/quality trade-off.

If asked White-box vs Black-box:

White-box = structural, tests the **code** (control flow + data flow), done by **developers**, early/small units. **Black-box** = functional, tests **external behaviour** (input→output), done by **professional testers**, late/large systems. Gray-box = mix of both. Example: checking every if/else branch = white-box; typing a password and checking the result = black-box.

(Also know: Test plan = high-level strategy · Test case = one input + expected result · Test suite = collection of cases run until stopping criteria.)